KavachX AI/ML Technical Assessment

Candidate Brief & Engineering Challenge Specifications

1. Welcome & Overview

Thanks for applying to KavachX. We turn existing CCTV cameras into an active safety surveillance system using AI models that run locally and send only verified events to the cloud. This task contains three core assignments based on real-world problems we solve daily.

> **Deadline:** Submit within 4 days (Tuesday) of receiving this brief.
>
> **Expected Effort:** Approximately 6–8 hours total (maximum 10 hours). A clear partial solution with insightful notes is preferred over a rushed complete implementation.
>
> **Setup & Environment:** Model training requires a GPU (a free Google Colab T4 instance is sufficient). All other code runs on a standard CPU laptop. No physical edge device or SSH connection is required for this assessment.

2. Suggested Time Breakdown

Task	Description	Suggested Time
Task 1	Plain-English rule to event JSON	3–4 hours
Task 2	Train, build model quantization pipeline, & fit edge budget	4 hours
Task 3	Cascaded false-positive verifier	2 hours

3. Guidelines & Rules

- **Environment:** Use Python 3.10 or newer.
- **Resources:** Documentation, Google, open-source libraries, and AI coding assistants are allowed. Please state in NOTES.md where an AI assistant was used; during the technical review call, you must be able to explain and modify any part of your code.
- **Integrity:** Complete the work independently.
- **Pragmatism:** Working code beats perfect code. If you skip or simplify anything, document what and why in NOTES.md.
- **Privacy:** Keep your repository private and do not share sample footage or datasets publicly.

4. Task 1: Plain-English Rule to Event JSON

KavachX clients set up monitoring by typing rules in natural language. For example:
"Alert me if a person carrying a bag enters the loading bay and stays more than 10 seconds."

Using clips/loading_bay.mp4 and the zone polygon in zones.json, build a script that:

1. **Rule Parsing:** Converts the plain-English rule into a structured configuration (target class, attribute, zone, dwell time threshold). A lightweight LLM call or rule parser is fine; hardcoding values is not allowed.
2. **Open-Vocabulary Detection:** Detects targets using an open-vocabulary model (e.g., YOLO-World via Ultralytics, OWL-ViT, or CLIP crop scoring) so new nouns (e.g., "forklift", "helmet") work without retraining.
3. **Tracking & Dwell Time:** Tracks individuals across frames and measures total time spent inside the defined polygon zone.
4. **Privacy Protection:** Blurs face and head regions on the evidence frame.
5. **Event Emission:** Outputs one JSON event per rule violation matching event_schema.json.

Event Schema (event_schema.json):

```
{
  "event_id": "uuid",
  "device_id": "kx-edge-001",
  "camera_id": "cam-03",
  "rule_id": "loading_bay_bag_dwell",
  "rule_text": "original English rule",
  "ts_start": "ISO-8601",
  "ts_end": "ISO-8601",
  "track_id": 17,
  "label": "person",
  "attributes": {"carrying": "bag"},
  "confidence": 0.0,
  "dwell_s": 0.0,
  "zone": "loading_bay",
  "evidence_frame": "base64 JPEG, faces masked",
  "needs_verification": true
}
```

Note: Set needs_verification to true when detection confidence is borderline. You will use this in Task 3.

5. Task 2: Train, Quantize, and Fit Edge Budget

For factory PPE compliance, we run quantized INT8 models locally. The dataset data/ppe/ contains ~300 CCTV frames in YOLO format (240 train / 60 val) with two classes: helmet and no_helmet.

1. **Train Model:** Fine-tune YOLOv8n from COCO weights at 416 px resolution for ~15 epochs on Google Colab. Choose appropriate data augmentations for CCTV angles and document your reasoning in NOTES.md.

2. **Evaluate Baseline:** Report mAP@50 and per-class recall on the validation set. Document 5 failure cases with explanations.

3. **Build Quantization Pipeline:** Write an automated model quantization pipeline script that exports the trained model to ONNX and generates FP32, FP16, and INT8 variants. For INT8, implement static post-training quantization (PTQ) using ~100 calibration images (e.g., via onnxruntime.quantization).

4. **Compare Performance:** Fill out the benchmark table below. If the INT8 quantized model drops more than 2 mAP points compared to FP32, test one mitigation technique and report results.

5. **Fit Edge Budget (Motion Gating Simulation):** Run the quantized INT8 model on clips/static_heavy.mp4 behind a motion gate (trigger detector execution only when movement occurs, with a full-frame pass every N frames). Measure and report detector calls per second with vs. without motion gating.

Benchmark Table Template:

Model Variant	Size (MB)	CPU Latency (ms)	mAP@50	no_helmet Recall
FP32 ONNX				
FP16 ONNX				
INT8 ONNX				
INT8 + Motion Gate				

6. Task 3: Cascaded False-Positive Verifier

To avoid sending unnecessary false alarms to customers, a second-stage verification filter evaluates flagged alerts. The folder alerts/ contains 30 nighttime intrusion snapshots labeled in labels.csv (true intrusion vs. false alarm like shadows, rain, or animal movement).

1. **Build Verifier:** Write a secondary verification script that scores each snapshot. Zero-shot CLIP with engineered text prompts or a hosted Vision-Language Model (VLM) API is acceptable.

2. **Evaluate Performance:** Calculate precision and recall for: (a) Edge Model Alone (all 30 alerts sent), and (b) Edge + Secondary Verifier. Select and justify an optimal operating confidence threshold.

3. **Economic & Product Analysis:** In NOTES.md, answer the following in ~0.5 pages (no code required):
   • A deployment site generates 200 alerts per day. What is the estimated monthly API cost for your verification setup?

- How would a "Flag as incorrect" button on the customer dashboard feed back into the system to improve verification over time?

7. Submission Guidelines & Repository Requirements

Create a private GitHub repository (grant access to the reviewer email provided) or submit a zip archive containing showcase-ready code:

1. **README.md:** Clear setup instructions and exact commands to execute each task script in a clean environment.
2. **Codebase & Pipeline Scripts:** Working scripts for Task 1, Task 2 (including the reproducible quantization pipeline), and Task 3. Include a pinned requirements.txt.
3. **Artifacts & Outputs:** Task 1 sample JSON & 2-3 masked frames; Task 2 results table, weights link, logs & plots; Task 3 precision/recall metrics & threshold rationale.
4. **NOTES.md:** Your overall design choices, trade-offs, AI assistant usage notes, Task 3 written response, and what you would improve given one additional week.
5. **Optional Video Walkthrough:** A brief (3–5 minute) video walk-through of your code running.